Detecting Backdoor Attacks with Hardware Performance Counters

Summary

This submission treats the hackathon task as **clean-only anomaly detection**. The supplied training file contains normal model inferences but no labeled backdoor examples. The detector learns a robust profile of the clean hardware-performance counters (HPCs), assigns every new inference an anomaly score, and labels unusually distant rows as possible `backdoor` cases.

The final model was trained on 800 clean rows with these counters: `cache-references`, `cycles`, and `LLC-loads`. Its alert threshold is **19.742107**, the **99.5th percentile** of the clean training scores.

Why HPCs are useful

Hardware performance counters are CPU measurements collected while a program runs. They include events such as cache activity and processor cycles. A backdoor can change the work performed during an inference—for example, by executing an additional trigger check or taking a different computation path. Those changes may leave a measurable execution footprint even when the model's ordinary output looks unchanged. HPCs therefore provide a low-level signal for detecting behavior that is unusual relative to clean executions.

The counters are not a security proof: operating-system activity, hardware variation, input complexity, and measurement noise can also change them. The detector uses them as evidence of unusual behavior.

Method

The program is implemented in `detector.py` using only Python's standard library:

1. Read the numeric HPC columns from a clean CSV.
2. Estimate each counter's normal centre with its **median**.
3. Estimate each counter's spread with `1.4826 × MAD` (median absolute deviation). This robust scale is less sensitive to extreme observations than a mean and standard deviation. A population-standard-deviation fallback is used for a flat counter.
4. For a row with counter values x_i, centre m_i, and scale s_i, calculate:

$$\text{score}(x) = \sum_i \left(\frac{x_i-m_i}{s_i}\right)^2$$

1. Set the threshold to the 99.5th percentile of the clean training scores. A new row is labeled `backdoor` when `score > threshold`; otherwise it is labeled `clean`.

The squared standardized distance puts counters on comparable scales and combines their evidence into one score. The high percentile is intended to keep the expected clean alert rate

low when the future clean distribution is similar to training.

Reproduction

From the repository root:

`bash python3 hackathon_solution/detector.py train \ --input attached_assets/trace_1788529753382.csv \ --model hackathon_solution/model.json

```
To score a validation file:

```bash
python3 hackathon_solution/detector.py predict \
  --input validation.csv \
  --model hackathon_solution/model.json \
  --output predictions.csv
```

No third-party packages are required. The output records the source `row_number`, the numeric `anomaly_score`, and a `prediction` of `clean` or `backdoor`.

Limitations and next steps

The clean file alone cannot measure recall or distinguish a real backdoor from an unusual but legitimate execution. The threshold is a clean-distribution calibration choice, not a learned attack boundary. If clean conditions drift, the false-positive rate can change.

The current score treats rows independently and does not model correlations between counters, sequence information, input metadata, or repeated-run variation. It also uses only the three supplied counters. Labeled organizer validation data should be used to select the threshold against the required metric and to measure accuracy, TPR, FPR, F1, and AUROC. The final predictions should then be exported in the organizers' required schema.
```